

ДИСЦИПЛИНА	Технологии индустриального программирования
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическое занятие
ПРЕПОДАВАТЕЛЬ	Адышкин Сергей Сергеевич
СЕМЕСТР	2 семестр, 2025-2026 гг.

Практическое занятие №2

gRPC: создание простого микросервиса, вызовы методов

Цель занятия

Освоить разработку простого микросервиса на Go с использованием gRPC, включая описание контракта в формате Protocol Buffers, генерацию кода, запуск gRPC-сервера и выполнение клиентских вызовов методов.

Задачи занятия

1. Изучить назначение gRPC и отличия RPC-подхода от привычного HTTP JSON API.
2. Освоить базовую структуру .proto-файла: сообщения, сервисы и методы.
3. Научиться генерировать Go-код из protobuf-описания с помощью protoc, protoc-gen-go и protoc-gen-go-grpc.
(gRPC)
4. Реализовать простой gRPC-сервер на Go.
5. Реализовать gRPC-клиент, вызывающий методы сервера.
6. Научиться передавать и принимать структурированные данные через protobuf-сообщения.
7. Отработать запуск, проверку и анализ работы gRPC-микросервиса на учебном примере.

Теоретическая часть

1. Что такое gRPC

gRPC – это фреймворк удалённого вызова процедур, который позволяет одному приложению вызывать методы другого приложения так, как будто это обычные локальные методы. В отличие от классического REST-подхода, где обычно используется HTTP и JSON, в gRPC интерфейс сервиса описывается заранее в .proto-файле, а затем на его основе генерируется код клиента и сервера. Официальная документация gRPC описывает этот подход именно как модель, где сервис определяется в proto-контракте, после чего генерируются привязки для клиента и сервера. (gRPC)

Для backend-разработки это важно потому, что взаимодействие между сервисами становится более формальным и строго типизированным. Если в REST API структура запроса и ответа часто описывается вручную в документации, то в gRPC контракт является частью кода и основой взаимодействия.

2. Почему gRPC часто используют в микросервисах

gRPC особенно популярен во внутренних взаимодействиях между сервисами, потому что:

- интерфейс строго описан;
- структуры данных типизированы;
- код клиента и сервера генерируется автоматически;
- protobuf обычно компактнее JSON;
- взаимодействие организуется вокруг методов сервиса, а не вокруг URL-маршрутов.

Эти особенности прямо связаны с тем, что Protocol Buffers предназначены для компактной сериализации структурированных данных, а gRPC использует protobuf как типичный IDL и формат сообщений. (protobuf.dev)

При этом gRPC не заменяет всё подряд. Во внешних публичных API часто по-прежнему удобен REST, потому что его проще тестировать из браузера и обычных HTTP-клиентов. Но для связи backend-сервисов внутри системы gRPC часто оказывается удобнее.

3. Роль Protocol Buffers

Protocol Buffers – это язык описания сообщений и сервисов, а также механизм генерации кода для разных языков. Сначала разработчик описывает контракт в .proto-файле: какие структуры данных существуют, какие поля в них входят, какие сервисы и методы доступны. Затем protoc создаёт код, который используется в приложении. Официальная документация protobuf подчёркивает, что .proto описывает формат данных, а компилятор создаёт код доступа к этим данным для конкретного языка. (protobuf.dev)

Это важный принцип: сначала контракт, потом реализация. Не наоборот.

4. Что мы создадим в этой работе

В данной практике будет реализован простой учебный gRPC-сервис **StudentService** с двумя методами:

- `GetStudentByID` – возвращает информацию о студенте по идентификатору;
- `Ping` – возвращает простой ответ сервера, чтобы убедиться, что сервис работает.

На стороне сервера будет храниться небольшой набор данных в памяти. На стороне клиента будет выполнено подключение к gRPC-сервису и вызов этих методов.

5. Чем gRPC отличается от REST на уровне разработки

При REST-подходе разработчик обычно:

- вручную проектирует URL;
- вручную описывает JSON;
- вручную пишет обработчики и документацию.

При gRPC-подходе разработчик:

- описывает контракт в `.proto`;
- генерирует код;
- реализует методы сервиса;
- использует сгенерированный типизированный клиент.

Это снижает вероятность рассинхронизации между сервером и клиентом, но требует дополнительного шага генерации кода и работы с protobuf-инфраструктурой.

Практическая часть

Общая идея проекта

В этой работе создаётся один gRPC-сервис и один клиент для его проверки.

Сервис будет слушать порт 50051.

Клиент будет подключаться к сервису и вызывать методы:

- Ping
- GetStudentByID

Рекомендуемая структура проекта

```
pz2-grpc/  
  proto/  
    student.proto  
  cmd/  
    server/  
      main.go  
    client/  
      main.go  
  internal/  
    student/  
      service.go  
      data.go  
  gen/  
    studentpb/  
      ... generated files ...  
go.mod
```

Шаг 1. Создание проекта

Откройте PowerShell и создайте папку проекта:

```
mkdir pz2-grpc
```

```
cd pz2-grpc
go mod init example.com/pz2-grpc
mkdir proto
mkdir cmd
mkdir cmd\server
mkdir cmd\client
mkdir internal
mkdir internal\student
mkdir gen
```

Для macOS/Linux:

```
mkdir -p pz2-grpc/proto
mkdir -p pz2-grpc/cmd/server
mkdir -p pz2-grpc/cmd/client
mkdir -p pz2-grpc/internal/student
mkdir -p pz2-grpc/gen
cd pz2-grpc
go mod init example.com/pz2-grpc
```

Шаг 2. Установка зависимостей Go

В корне проекта выполните:

```
go get google.golang.org/grpc
go get google.golang.org/protobuf
```

Шаг 3. Установка инструментов генерации

Установите плагины для генерации Go-кода из .proto:

```
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest
```

Эти команды соответствуют официальным инструкциям для генерации Go-кода из protobuf и gRPC-описаний. В документации protobuf указано использование protoc-gen-go, а в документации gRPC для Go – protoc-gen-go-grpc. ([gRPC](#))

Важно: папка с установленными бинарными файлами должна быть доступна в PATH, иначе protoc не сможет найти плагины. Это также отражено в официальной документации protobuf для Go. ([protobuf.dev](#))

Шаг 4. Установка protoc

На компьютере должен быть установлен protoc. Его наличие можно проверить командой:

```
protoc --version
```

Если команда обрабатывается успешно, значит компилятор установлен.

Шаг 5. Создание proto-контракта

Создайте файл proto/student.proto:

```
syntax = "proto3";

package student;

option go_package = "example.com/pz2-grpc/gen/studentpb;studentpb";

message PingRequest {
    string message = 1;
}
```



```
message PingResponse {
    string message = 1;
}

message GetStudentRequest {
    int64 id = 1;
}

message Student {
    int64 id = 1;
    string full_name = 2;
    string group = 3;
    string email = 4;
}

message GetStudentResponse {
    Student student = 1;
}

service StudentService {
    rpc Ping(PingRequest) returns (PingResponse);
    rpc      GetStudentByID(GetStudentRequest)      returns
(GetStudentResponse);
}
```

Пояснение

В этом файле:

- описаны сообщения запроса и ответа;
- описана сущность Student;
- описан сервис StudentService;

- описаны два RPC-метода.

`option go_package` указывает, в какой Go-пакет должен генерироваться код. Принципы размещения и генерации Go-кода из `.proto` описаны в руководстве по generated code для Go. (protobuf.dev)

Шаг 6. Генерация кода

Выполните команду из корня проекта:

```
protoc --proto_path=proto --go_out=. --go-grpc_out=. proto/student.proto
```

После этого должны появиться сгенерированные файлы в соответствии с `go_package`.

Если генерация прошла успешно, в проекте появятся файлы вида:

- `student.pb.go`
- `student_grpc.pb.go`

Официальная документация `protobuf` и `gRPC` для Go описывает именно такой подход: `.proto` подаётся в `protoc`, а вывод управляется флагами `--go_out` и `--go-grpc_out`. ([gRPC](https://grpc.io))

Шаг 7. Данные сервиса

Создайте файл `internal/student/data.go`:

```
package student

import (
    "errors"

    "example.com/pz2-grpc/gen/studentpb"
)
```

```
var ErrStudentNotFound = errors.New("student not found")

type Repository struct {
    data map[int64]*studentpb.Student
}

func NewRepository() *Repository {
    return &Repository{
        data: map[int64]*studentpb.Student{
            1: {
                Id:          1,
                FullName:    "Иванов Иван Иванович",
                Group:       "ИБО-01-25",
                Email:       "ivanov@example.com",
            },
            2: {
                Id:          2,
                FullName:    "Петрова Мария Сергеевна",
                Group:       "ИБО-02-25",
                Email:       "petrova@example.com",
            },
            3: {
                Id:          3,
                FullName:    "Сидоров Алексей Андреевич",
                Group:       "ИБО-03-25",
                Email:       "sidorov@example.com",
            },
        },
    }
}
```

```

func (r *Repository) GetByID(id int64)
(*studentpb.Student, error) {
    st, ok := r.data[id]
    if !ok {
        return nil, ErrStudentNotFound
    }
    return st, nil
}

```

Шаг 8. Реализация gRPC-сервиса

Создайте файл `internal/student/service.go`:

```

package student

import (
    "context"

    "example.com/pz2-grpc/gen/studentpb"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"
)

type Service struct {
    studentpb.UnimplementedStudentServiceServer
    repo *Repository
}

func NewService(repo *Repository) *Service {
    return &Service{repo: repo}
}

```

```

func (s *Service) Ping(ctx context.Context, req
*studentpb.PingRequest) (*studentpb.PingResponse, error) {
    msg := req.GetMessage()
    if msg == "" {
        msg = "ping"
    }

    return &studentpb.PingResponse{
        Message: "Server received: " + msg,
    }, nil
}

func (s *Service) GetStudentByID(ctx context.Context,
req *studentpb.GetStudentRequest)
(*studentpb.GetStudentResponse, error) {
    id := req.GetId()
    if id <= 0 {
        return nil, status.Error(codes.InvalidArgument,
"invalid student id")
    }

    st, err := s.repo.GetByID(id)
    if err != nil {
        return nil, status.Error(codes.NotFound, "student
not found")
    }

    return &studentpb.GetStudentResponse{
        Student: st,
    }, nil
}

```

Пояснение

Здесь реализуются методы сервиса, объявленные в `.proto`.
Важный момент: структура сервиса встраивает `UnimplementedStudentServiceServer`, который генерируется плагином `gRPC` для `Go` и используется в типичном шаблоне реализации серверов. Это отражено в материалах по `gRPC Go` и генератору `protoc-gen-go-grpc`. ([gRPC](#))

Шаг 9. Точка входа gRPC-сервера

Создайте файл `cmd/server/main.go`:

```
package main

import (
    "log"
    "net"

    "example.com/pz2-grpc/gen/studentpb"
    "example.com/pz2-grpc/internal/student"
    "google.golang.org/grpc"
)

func main() {
    lis, err := net.Listen("tcp", ":50051")
    if err != nil {
        log.Fatal(err)
    }

    repo := student.NewRepository()
    service := student.NewService(repo)
```

```
server := grpc.NewServer()
studentpb.RegisterStudentServiceServer(server,
service)

log.Println("gRPC server started on :50051")

if err := server.Serve(lis); err != nil {
    log.Fatal(err)
}
}
```

Шаг 10. Реализация gRPC-клиента

Создайте файл `cmd/client/main.go`:

```
package main

import (
    "context"
    "log"
    "time"

    "example.com/pz2-grpc/gen/studentpb"
    "google.golang.org/grpc"
    "google.golang.org/grpc/credentials/insecure"
)

func main() {
    conn, err := grpc.NewClient(
        "localhost:50051",

        grpc.WithTransportCredentials(insecure.NewCredentials
    )),
```

```

    )
    if err != nil {
        log.Fatal(err)
    }
    defer conn.Close()

    client := studentpb.NewStudentServiceClient(conn)

    ctx,                                cancel                                :=
context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    pingResp,      err      :=      client.Ping(ctx,
&studentpb.PingRequest{
        Message: "hello grpc",
    })
    if err != nil {
        log.Fatal("Ping error:", err)
    }
    log.Println("Ping response:", pingResp.GetMessage())

    studentResp,   err   :=   client.GetStudentByID(ctx,
&studentpb.GetStudentRequest{
        Id: 1,
    })
    if err != nil {
        log.Fatal("GetStudentByID error:", err)
    }

    st := studentResp.GetStudent()
    log.Printf("Student:  id=%d,  full_name=%s,  group=%s,
email=%s\n",

```



```
    st.GetId(),  
    st.GetFullName(),  
    st.GetGroup(),  
    st.GetEmail(),  
)  
}
```

Пояснение

Для подключения здесь используется небезопасный `transport credential`, что допустимо для локальной учебной среды. В реальных системах соединение обычно защищают TLS. В официальных примерах gRPC для Go используется подобный режим для локального quick start и демонстрационных сценариев. ([gRPC](#))

Шаг 11. Запуск сервера

Откройте терминал в корне проекта и выполните:

```
go run ./cmd/server
```

Ожидаемый результат:

```
gRPC server started on :50051
```

Шаг 12. Запуск клиента

Откройте второе окно PowerShell и выполните:

```
go run ./cmd/client
```

Ожидаемый пример вывода:

```
Ping response: Server received: hello grpc  
Student: id=1, full_name=Иванов Иван Иванович,  
group=ИББО-01-25, email=ivanov@example.com
```

Шаг 13. Проверка сценария ошибки

Измените в клиенте:

```
Id: 1,
```

на

```
Id: 999,
```

и снова запустите клиента.

Ожидается ошибка вида `NotFound`, потому что сервер не найдёт студента с таким идентификатором. В реализации выше это возвращается через `status.Error(codes.NotFound, ...)`, то есть как типичная gRPC-ошибка с кодом статуса.

Шаг 14. Что получилось в результате

В этой работе студент создал полноценный минимальный gRPC-микросервис:

- описал контракт сервиса в `.proto`;
- сгенерировал код клиента и сервера;
- реализовал gRPC-сервер на Go;
- реализовал gRPC-клиент на Go;
- выполнил вызовы методов сервиса;
- получил структурированные ответы;
- обработал типовую ошибку.

Что важно понять по итогам работы

gRPC меняет сам стиль разработки межсервисного взаимодействия. При таком подходе API строится не вокруг URL и JSON, а вокруг методов сервиса и формального контракта в `.proto`.

Это даёт несколько важных преимуществ:

- меньше ручной рутины при написании клиента;

- строгая типизация;
- меньше риска расхождения контракта между сервером и клиентом;

- лучше подходит для внутреннего взаимодействия сервисов.

Но появляются и новые требования:

- нужно уметь работать с `proto`;
- нужно следить за генерацией кода;
- локальная отладка может быть менее привычной, чем обычный HTTP JSON API;

- контракт нужно проектировать аккуратно заранее.

Дополнительные задания

Вариант 1. Добавить метод получения списка студентов

Добавьте в `.proto` новый метод:

```
rpc ListStudents(google.protobuf.Empty) returns
(ListStudentsResponse);
```

И реализуйте соответствующий ответ в сервере.

Вариант 2. Добавить создание студента

Добавьте метод:

```
rpc CreateStudent(CreateStudentRequest) returns
(GetStudentResponse);
```

и реализуйте добавление в память.

Вариант 3. Добавить поле **specialization**

Измените структуру `Student` в `.proto`, затем заново сгенерируйте код и обновите серверную реализацию.

Вариант 4. Сравнить с REST

Кратко опишите, как тот же сервис выглядел бы в формате REST API: какие были бы URL, какие JSON-запросы и какие ответы.

Контрольные вопросы

1. Что такое gRPC?
2. Какую роль играет `.proto`-файл?
3. Для чего нужен `protoc`?
4. Зачем используются `protoc-gen-go` и `protoc-gen-go-grpc`? (protobuf.dev)
5. Чем gRPC отличается от HTTP JSON API?
6. Почему контракт в gRPC считается строго типизированным?
7. Что делает gRPC-клиент в этой работе?
8. Что происходит, если клиент запрашивает несуществующего студента?
9. Почему для локальной учебной среды допустимо использовать `insecure credentials`?
10. В каких случаях gRPC особенно удобен в backend-разработке?

Типичные ошибки

Если `protoc` не находит `protoc-gen-go` или `protoc-gen-go-grpc`, обычно проблема в том, что путь к бинарным файлам не добавлен в `PATH`.

Если не генерируются файлы в нужную папку, нужно проверить `option go_package` в `.proto`.

Если клиент не подключается к серверу, сначала проверьте, запущен ли сервер и совпадает ли порт 50051.

Если сервер не компилируется, убедитесь, что после изменения `.proto` вы заново выполнили генерацию кода.

Если метод возвращает ошибку `NotFound`, это может означать, что в памяти сервера нет объекта с таким ID.